

ALBERI BINARI

INDICE SEZIONE

- ☐ DEFINIZIONE
- ☐ INTERFACCIA
- ☐ PROPRIETA' DEGLI ALBERI BINARI PROPRI
- ☐ ALBERI BINARI PROPRI ESTREMI
- ☐ VISITE DI ALBERI BINARI
- ☐ ALBERI BINARI COMPLETI
- ☐ IMPLEMENTAZIONE CON ARRAY

DEFINIZIONE

ALBERO BINARIO

Un *albero binario* T è un albero ordinato in cui:

- Ogni nodo *interno* ha ≤ 2 figli.
- Ogni nodo *non radice* è etichettato come figlio *sinistro* o *destro* di suo padre.
- Se ci sono entrambi i figli, il *sinistro viene prima* del destro nell'ordinamento dei figli di un nodo.

Si definiscono anche gli alberi binari **propri**:

ALBERO BINARIO PROPRIO

Un *albero binario proprio* T è un albero binario tale che ogni nodo interno ha *esattamente due figli*.

Spesso gli alberi binari propri (*proper binary trees*) sono chiamati anche **pieni** (*full binary trees*).

INTERFACCIA

L'interfaccia BinaryTree eredita chiaramente tutti i metodi dell'interfaccia Tree, ai quali aggiunge:

```
public interface BinaryTree<E> extends Tree<E>{

    // Returns the Position of p's left child (or null if it doesn't exist)
    Position<E> left(Position<E> p);

    // Returns the Position of p's right child (or null if it doesn't exist)
    Position<E> right(Position<E> p);

    // Returns the Position of p's sibling (or null if no sibling exists)
    Position<E> sibling(Position<E> p);

}
```

PROPRIETA' DEGLI ALBERI BINARI PROPRI

Sia T un albero binario proprio.

Chiamiamo n il *numero di nodi* in T ed m il *numero di foglie* in T .

Allora il *numero di nodi interni* in T è $n - m$.

Chiamiamo inoltre h l'*altezza* di T .

Valgono allora le seguenti proprietà:

🔗 PROPRIETA' DI UN ALBERO BINARIO

1. $m = n - m + 1$
2. $h + 1 \leq m \leq 2^h$
3. $h \leq n - m \leq 2^h - 1$
4. $2h + 1 \leq n \leq 2^{h+1} - 1$
5. $\log_2(n + 1) - 1 \leq h \leq \frac{n-1}{2}$

DIMOSTRAZIONI

✓ PROP. 1 $m = n - m + 1$

Procediamo per induzione sull'altezza $h \geq 0$ di T .

BASE: Per $h = 0$ abbiamo $m = 1 - 1 + 1 = 1$ vero.

PASSO INDUTTIVO: Fissiamo $h \geq 0$ arbitrario.

IP INDUTTIVA: assumiamo che la proprietà valga per alberi di altezza $h' \leq h$.

Sia T un albero binario proprio di altezza $h + 1 \geq 1$ e dimostriamo che la proprietà vale anche per T .

Per farlo, immaginiamo T come due alberi binari propri T_1 e T_2 di altezze $h_1, h_2 \leq h$

uniti da una nuova radice.

$$h + 1 = 1 + \max\{h_1, h_2\} \implies h_1, h_2 \leq h$$

E quindi per T_1 e T_2 vale l'ipotesi induttiva.

A questo punto sia m_i il numero di foglie di T_i ed n_i il numero di nodi di T_i per $i = 1, 2$.

Abbiamo il numero di foglie di T :

$$m = m_1 + m_2$$

E anche il numero totale di nodi di T :

$$n = n_1 + n_2 + 1$$

Applicando l'ipotesi induttiva ai sottoalberi:

$$m = (n_1 - m_1 + 1) + (n_2 - m_2 + 1) = (n_1 + n_2 + 1) - (m_1 + m_2) + 1 = n - m + 1$$

E concludiamo la dimostrazione. $\square$

NOTA

In un albero binario proprio T , vale quindi $n = 2m - 1$, cioè il numero totale di nodi è dispari.

✓ **PROP. 2** $h + 1 \leq m \leq 2^h$

Cominciamo dimostrando che $m \leq 2^h$ per induzione su $h \geq 0$.

BASE: $h = 0$ abbiamo $m = 1$ e la proprietà è vera.

PASSO INDUTTIVO: Fissiamo $h \geq 0$ arbitrario.

IP INDUTTIVA: la proprietà vale per tutti gli alberi binari propri di altezza $\leq h$.

Sia T un albero binario proprio di altezza $h + 1 \geq 1$

Immaginiamo ancora una volta T come due alberi binari propri T_1, T_2 di altezze $h_1, h_2 \leq h$ uniti da una nuova radice.

Per T_1 e T_2 vale l'ipotesi induttiva.

A questo punto sia m_i il numero di foglie di T_i ed n_i il numero di nodi di T_i per $i = 1, 2$.

Abbiamo allora:

$$m = m_1 + m_2 \leq 2^{h_1} + 2^{h_2}$$

Per ipotesi induttiva.

Possiamo dire allora:

$$m \leq 2^{h_1} + 2^{h_2} = 2^{h+1}$$

E concludiamo la dimostrazione della seconda disuguaglianza.

Dimostriamo ora la prima, considerando ancora T_1 e T_2 .

$$m = m_1 + m_2 \geq h_1 + 1 + h_2 + 1$$

A questo punto osserviamo che:

$$h_1 + h_2 + 2 \geq 1 + \max\{h_1, h_2\} = h + 1$$

Allora concludiamo che vale anche la prima disuguaglianza:

$$m \geq h + 1$$

E abbiamo terminato la dimostrazione. $\square$

✓ **PROP. 3** $h \leq n - m \leq 2^h - 1$

Per dimostrarla è sufficiente sostituire in $P2$ m con $n - m + 1$ (da $P1$).

Si ottiene infatti:

$$h + 1 \leq n - m + 1 \leq 2^h$$

Da cui troviamo:

$$h \leq n - m \leq 2^h - 1$$

E concludiamo. $\square$

✓ **PROP. 4** $2h + 1 \leq n \leq 2^{h+1} - 1$

Similmente, otteniamo $P4$ sommando $P2$ e $P3$.

$$2h + 1 \leq n \leq 2 \cdot 2^h - 1 = 2^{h+1} - 1$$

E concludiamo. $\square$

✓ **PROP. 5** $\log_2(n + 1) - 1 \leq h \leq \frac{n-1}{2}$

Possiamo anche ottenere la quinta a partire da $P4$.

Dalla prima disuguaglianza:

$$2h + 1 \leq n \implies h \leq \frac{n-1}{2}$$

E dalla seconda:

$$2^{h+1} - 1 \geq n \implies h \geq \log_2(n + 1) - 1$$

E unendo le due disuguaglianze trovate concludiamo la dimostrazione. $\square$

ALBERI BINARI PROPRI ESTREMI

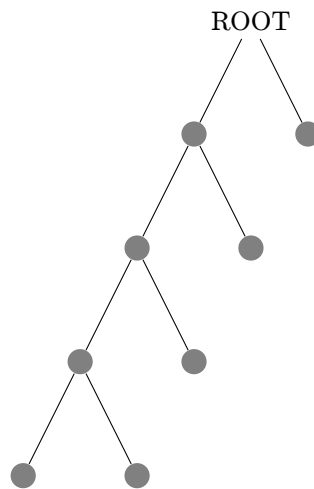
🔗 NOTA

La $P5$ implica che in un albero binario proprio con n nodi, l'altezza è compresa tra $\Omega(\log n)$ e $O(n)$.

Questi *boundries* rappresentano le seguenti situazioni.

1. Albero radicato *solo verso sinistra* (sarebbe equivalente a destra).

Abbiamo $h = \frac{n-1}{2}$ o equivalentemente $n = 2h + 1$.



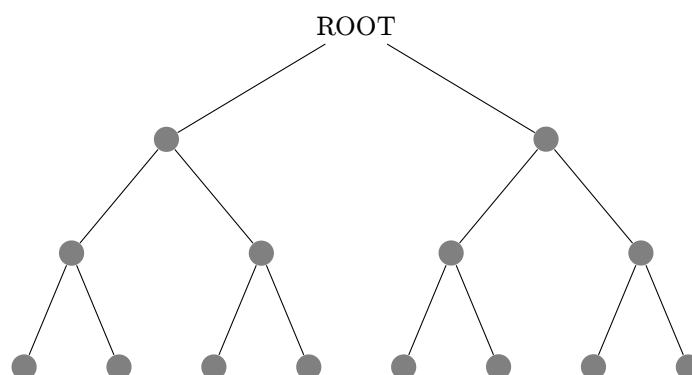
⚠️ NOTA BENE

In assenza di altre ipotesi, il migliore upper bound all'altezza di un albero binario con n nodi è $O(n)$.

NON $O(\log n)$.

2. Albero *"completamente radicato"*.

Abbiamo $h = \log_2(n + 1) - 1$ o equivalentemente $n = 2^{h+1} - 1$.



VISITE DI ALBERI BINARI

Oltre alle visite in *preorder* e *postorder*, per gli alberi binari si definisce anche la vista *inorder*, basata sulla regola seguente:

VISITA INORDER

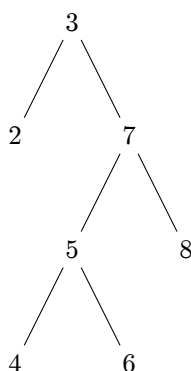
1. Prima (*ricorsivamente*) il sottoalbero sx.
2. Poi il **padre**.
3. Infine (*ricorsivamente*) il sottoalbero dx.

L'algoritmo è quindi il seguente:

- **INPUT:** $v \in T$
- **OUTPUT:** visita inorder di T_v

```
Algorithm inorder(v)
1: if T.left(v) ≠ null then
2:   INORDER(T.left(v))
3: end if
4: visita v
5: if T.right(v) ≠ null then
6:   INORDER(T.right(v))
7: end if
```

Consideriamo per esempio il seguente albero:



L'ordine di visita, eseguendo l'algoritmo a partire dalla radice, è: **2, 3, 4, 5, 6, 7, 8**.

OSSERVAZIONE

E' un esempio di **albero binario di ricerca** (che studieremo più avanti), dove la visita tocca gli elementi presenti in ordine crescente di valore.

COMPLESSITA'

La complessità è data da:

$$\Theta \left(n + \sum_{v \in T} t_v \right)$$

Dove n è il numero di nodi dell'albero (che vengono tutti scansionati) e t_v è il costo della visita del nodo v .

L'analisi è la stessa vista per alberi generali.

APPLICAZIONE: ESPRESSIONI ARITMETICHE

PARSE TREE

Il *parse tree* T associato ad un'espressione aritmetica E (con operatori solo binari) è un *albero binario proprio* in cui:

- I nodi **foglia** contengono le **costanti**/variabili di E .
- I nodi **interni** contengono gli **operatori** di E .

In modo tale che:

- Se $E = a$ con a costante/variabile, allora T è costituito da un'unica foglia contenente a .
- Se $E = (E_1 \mathbf{Op} E_2)$, la radice di T contiene **Op** e ha come sottoalbero sinistro (risp. destro) il parse tree associato a E_1 (risp. E_2).

Vediamo di seguito un esempio, dato dall'espressione:

$$E = \left((15 + x) \times (7 - (9 \div 3)) \right)$$

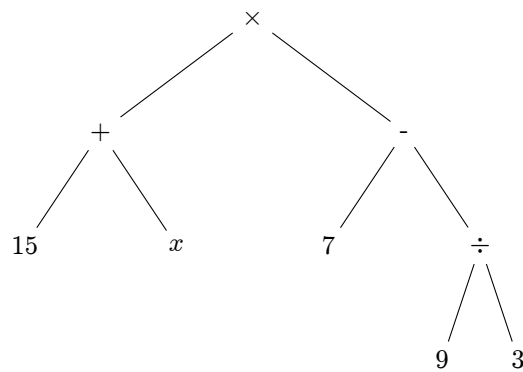
L'espressione così rappresentata (notazione *infissa*) è facilmente leggibile dall'uomo, ma per un'elaborazione da parte di un calcolatore conviene utilizzare la **notazione postfissa** (o *polacca inversa*), che consiste nello scrivere *prima gli operandi* e *poi gli operatori*.

Nel nostro caso abbiamo allora:

$$E = \underbrace{15x+793}_{\text{operandi}} \div - \times$$

Sopra sono stati evidenziati gli operandi della moltiplicazione per aiutare nella comprensione.

Il *parse tree* associato alla nostra espressione è:



🔥 OSSERVAZIONI

1. Le parentesi sono *implicite* nella struttura dell'albero.
2. I **compilatori** generano il parse tree *a partire* dall'espressione in notazione *infissa*, eseguono *type checking*, *verifiche* e *ottimizzazioni* sul parse tree, e poi ricavano l'espressione in notazione *postfissa*, che può essere calcolata molto velocemente mediante lo stack.

GENERAZIONE ESPRESSIONE IN NOTAZIONE INFISSA

Possiamo generare l'espressione in notazione infissa a partire dal parse tree con una visita **inorder**.

Vediamo l'algoritmo:

- **INPUT**: Parse tree T per E , $v \in T$, lista L .
- **OUTPUT**: Aggiunta a L di E_v in notazione infissa.

Algorithm infix(T, v, L)

```

1: if T.isExternal(v) then
2:   L.addLast(v.getElement())
3: else
4:   L.addLast('(')
5:   INFIX(T, T.left(v), L)
6:   L.addLast(v.getElement())
7:   INFIX(T, T.right(v), L)
8:   L.addLast('')
9: end if
  
```

Per quanto visto circa la complessità delle visite, e dato che il costo delle operazioni su un nodo v è $\Theta(1)$, abbiamo che la complessità è $\Theta(n)$.

GENERAZIONE ESPRESSIONE IN NOTAZIONE POSTFISSA

Possiamo generare l'espressione in notazione postfissa a partire dal parse tree con una visita **postorder**.

Vediamo l'algoritmo:

- **INPUT**: Parse tree T per E , $v \in T$, lista L .
- **OUTPUT**: Aggiunta a L di E_v in notazione postfissa.

Algorithm postfix(T, v, L)

```
1: if T.isExternal(v) then
2:   L.addLast(v.getElement())
3: else
4:   POSTFIX( $T, T.left(v), L$ )
5:   POSTFIX( $T, T.right(v), L$ )
6:   L.addLast(v.getElement())
7: end if
```

E ancora una volta la complessità è $\Theta(n)$.

ALBERI BINARI COMPLETI

📖 ALBERO BINARIO COMPLETO

Albero binario di altezza $h \geq 0$ tale che:

- $\forall i, 0 \leq i \leq h - 1$: il livello i ha 2^i nodi (il **massimo**).
- Al livello $h - 1$ tutti i nodi **interni** sono alla **sinistra** delle eventuali foglie e hanno tutti 2 **figli** tranne, eventualmente, quello più a destra che, se ha un solo figlio, ha il figlio sinistro.

🔥 PROPOSIZIONE

Un albero binario completo con n nodi ha altezza $h = \lfloor \log_2 n \rfloor$.

✓ DIM.

Sia T un albero binario completo di altezza h .

Al livello h , per definizione, il numero di nodi n_h è:

$$1 \leq n_h \leq 2^h$$

Per i livelli precedenti i , invece, è esattamente 2^i .

Allora abbiamo:

$$1 + \sum_{i=0}^{h-1} 2^i \leq n \leq 2^h \sum_{i=0}^{h-1} 2^i = \sum_{i=0}^h 2^i$$

Per cui troviamo:

$$1 + 2^h + 1 = 2^h \leq n \leq 2^{h+1} - 1 < 2^{h+1}$$

E quindi:

$$h \leq \log_2 n \leq h + 1$$

Che implica esattamente $h = \lfloor \log_2 n \rfloor$. $\square$

Tale fatto ha una conseguenza importante: $\forall n \geq 1$ esiste *un unico* albero binario completo di n nodi.

IMPLEMENTAZIONE CON ARRAY

Il seguente schema consente di mappare un albero binario su un array $P = P[0], P[1] \dots$

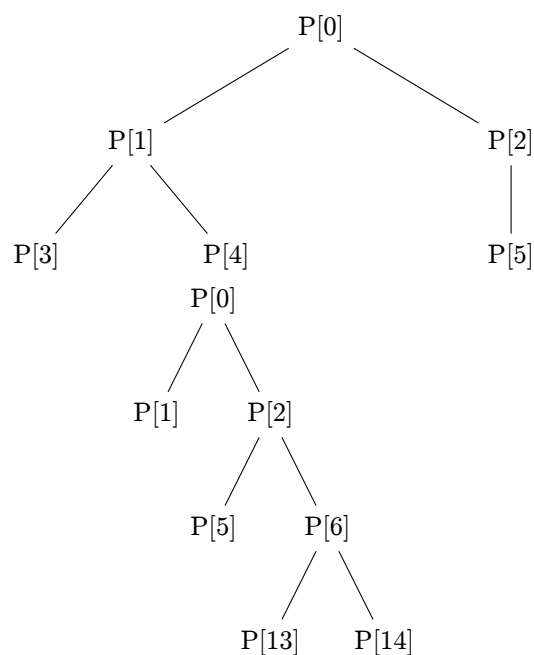
📖 LEVEL NUMBERING

- Radice $\rightarrow P[0]$.
- Figli di $P[i] \rightarrow P[2i + 1], P[2i + 2]$.
- Padre di $P[i] \rightarrow P \left[\left\lfloor \frac{i-1}{2} \right\rfloor \right]$.

🔥 OSSERVAZIONE

La rappresentazione è *space-efficient* per alberi *molto bilanciati*, ma non lo è affatto per alberi sbilanciati.

Consideriamo infatti i seguenti esempi:



Nel primo esempio i 6 nodi sono mappati negli indici $[0 \div 5]$, senza nessuno spazio lasciato libero.

Nel secondo esempio, invece, sono sprecate le posizioni di indici 3, 4, 7, 8, 9, 10, 11, 12.

ALBERI BINARI COMPLETI SU ARRAY

E' facile dimostrare che usando il level numbering per mappare un *albero binario completo* con $n \geq 1$ nodi e altezza h su un array P si ha che:

- Per ogni $0 \leq i \leq h$ i 2^i nodi del livello, presi da sinistra a destra, sono mappati in $P[2^i - 1], P[2^i], \dots, P[2^{i+1} - 2]$.
- I nodi del livello h , presi da sinistra verso destra, sono mappati in $P[2^h - 1], P[2^h], \dots, P[n - 1]$.
- Il nodo *last* è mappato in $P[n - 1]$.

La rappresentazione è quindi *space-efficient*.

Inoltre il level numbering definisce una *corrispondenza 1:1* tra *array* ed *alberi binari completi*.